# REVA University

# Major Project Report — Phase-1 Review (Academic Year 2026–2027)

**United Nations Sustainable Development Goals Alignment: Goal 9 (Industry, Innovation & Infrastructure) & Goal 11 (Sustainable Cities & Communities)**

## Student and Project Details

| Field | Details |
|---|---|
| Student name | RAGHUNATHAREDDY GR |
| SRN / Roll number | R23EA094 |
| Program and Section | B.Tech Computer Science & Engineering (AIML), 7th Sem (Academic Year 2026–2027) |
| Project title | Smart City IoT Intrusion Detection System with Sub-Millisecond Fog Triage |
| Faculty guide | Dr. Supreeth S, Associate Professor, School of C&SE, REVA University |
| Repository & Profile | GitHub: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY- LinkedIn: https://www.linkedin.com/in/raghunatha-reddy-gr |

## 1. Literature Survey and Requirement Analysis

### 1.1 Problem Statement and Context

Smart City municipal infrastructures connect tens of thousands of heterogeneous Internet of Things (IoT) edge telemetry devices—including intelligent traffic signaling arrays, SCADA potable water distribution monitoring nodes, environmental air quality sensors, and automated surveillance networks. These distributed nodes are persistently vulnerable to volumetric multi-vector cyber-attacks, specifically Distributed Denial-of-Service (DDoS), SYN flooding storms, and automated port scanning reconnaissance.

Target Users & Industrial Scope: The primary users of this system are Municipal Security Operations Centers

(SOC), smart city network infrastructure administrators, and edge gateway systems engineers. In conventional architectures, raw NetFlow telemetry is forwarded across wide-area networks (WAN) to distant centralized cloud servers for deep threat analysis. Under modern high-rate volumetric attacks, this approach introduces intolerable backhaul latency (150–400 ms) and rapidly exhausts uplink network bandwidth, creating catastrophic cascading blackouts across municipal control systems.

Engineering Objective: This module deploys directly at the municipal Fog Gateway level, executing autonomous, sub-millisecond packet triage (<100 µs per flow). By filtering >85% of benign traffic locally and isolating volumetric attacks within microseconds, it safeguards cloud bandwidth while delivering >99% multi-class attack detection accuracy.

## 1.2 Literature Survey and Comparative Benchmarking

A rigorous survey of recent literature across IEEE and ACM publications was performed to establish baseline performance metrics in edge intrusion detection, feature extraction schemas, and computational latency constraints. The surveyed literature was evaluated across canonical NetFlow feature sets, classification latency, and memory footprints as summarized in Table 2.

| Paper (author, year, venue) | Approach / technique | Strength | Limitation |
|---|---|---|---|
| Sarhan et al. (2023), IEEE TNSM | NetFlow standard feature extraction across IoT benchmark datasets | High cross-topology feature generalizability (46 canonical NetFlow features) | Offline batch processing only; lacked sub-millisecond edge streaming capabilities |
| Kumar et al. (2022), IEEE IoT Journal | Ensemble Deep Learning on Fog Gateways for SCADA security | High multi-vector detection accuracy (98.7% F1 on CICIoT benchmarks) | Heavy memory footprint (420 MB); 185 ms inference latency bottlenecks edge gateway nodes |
| Ferrag et al. (2021), IEEE IoT Magazine | Survey of deep learning & edge-cloud taxonomies for IoT | Comprehensive threat taxonomy covering 35 cyber-attack vectors | Demonstrated that raw cloud offloading saturates municipal backhaul bandwidth during DDoS bursts |

| Paper (author, year, venue) | Approach / technique | Strength | Limitation |
|---|---|---|---|
| Al-Hawawreh et al. (2023), Computers & Security | Lightweight decision tree ensembles for Industrial IoT gateways | Deterministic sub-100 µs execution and low memory consumption | Lacks hysteresis band; high false alarm churn under transient sensor noise fluctuations |

### 1.3 Research Gaps and Problem Formulation

Critical Gap 1: Latency-Accuracy Trade-off in Deep Learning IDS. While deep neural networks (CNNs, LSTMs, Transformers) achieve high theoretical classification accuracy, their memory footprint (>200 MB) and computational latency (150–280 ms) render them completely impractical for low-power, resource-constrained edge gateways.

Critical Gap 2: Flapping Alarms and Inflexibility of Static Rule Engines. Traditional network intrusion detection systems such as Snort and Suricata rely on rigid pattern matching. They fail to detect novel polymorphic zero-day variants and trigger excessive false positive alarms (12–25%) during natural burst fluctuations of IoT sensor traffic.

The Gap Addressed by My Module: Our system introduces a hybrid, dual-tier fog triage architecture. By pairing an ultra-compact Random Forest classifier (48 µs execution time, 1.4 MB memory) with a dual-threshold hysteresis filter [0.20, 0.80], the module deterministically resolves unambiguous flows locally while deterministically routing ambiguous edge cases to Tier-2 SOC infrastructure, entirely preventing false alarm cascades.

### 1.4 Requirement Specifications and Technical Constraints

System requirements were rigorously formulated based on real-world municipal Fog gateway performance benchmarks and operational security mandates. System requirements are categorized into functional triage and non-functional latency/throughput constraints in Table 3.

| ID | Requirement | Type (functional / non-functional) | Priority |
|---|---|---|---|
| R1 | Real-time NetFlow ingestion engine parsing >=1,000 flows/sec with circular ring buffering | Functional | High |
| R2 | Lightweight Random Forest edge classifier with sub-100 µs inference latency | Functional | High |

| ID | Requirement | Type (functional / non-functional) | Priority |
|---|---|---|---|
| | (achieved 48 µs) | | |
| R3 | Dual-threshold hysteresis filter [0.20, 0.80] for benign noise suppression & Tier-2 escalation | Functional | High |

## 2. System and Detailed Design

### 2.1 System Architecture and Topology

The system architecture follows a hierarchical three-tier topology consisting of: (1) IoT Sensor Layer, (2) Fog Gateway Triage Layer, and (3) Cloud Analytics & Storage Tier. At the Fog Gateway tier, incoming NetFlow packets are ingested through non-blocking asynchronous socket listeners into a pre-allocated circular ring buffer capable of sustaining 10,000 flows/sec.

Feature extraction standardizes raw packets into a 46-dimensional canonical NetFlow representation. Flows are then processed by the Scikit-Learn/ONNX Random Forest classifier. Clean benign traffic is passed forward to municipal controllers in 48 µs, malicious volumetric bursts are dropped instantly with forensic audit logging, and ambiguous borderline flows are queued for Tier-2 cloud escalation.

### 2.2 Detailed Architectural Design and Diagram Mapping

The system architecture is modeled across structural, behavioral, and data transformation perspectives to guarantee sub-millisecond execution and deterministic thread safety. Architectural interactions, flow logic, and data transformations are documented across three complementary engineering representations in Table 4 and Appendix A.

| Diagram | Shows |
|---|---|
| Flowchart | Main logic and decision flow: packet arrival, feature extraction, RF inference, dual-threshold hysteresis triage |
| UML (class / sequence / use case) | Structure and interactions: Sensor -> Gateway Parser -> Ring Buffer -> Triage Engine -> FastAPI -> Alert Bus |
| DFD (level 0 and 1) | Data flows between external IoT entities, transformation processes, feature cache, and SOC datastores |

### 2.3 Core Design Decisions and Trade-Off Analysis

Design Decision 1: Vectorized Random Forest vs Deep Neural Networks. We evaluated 1D-CNN and LSTM architectures against a 40-tree Random Forest. Although the CNN achieved 99.6% accuracy, its 185 ms inference latency and 420 MB footprint disqualified it from edge deployment. Our shallow Random Forest achieved 99.4% F1-score with an inference latency of just 48 µs and a 1.4 MB memory footprint.

Design Decision 2: Dual-Threshold Hysteresis Filter vs Single Threshold Argmax. Standard IDS employ a fixed 0.50 probability cutoff. However, noisy IoT telemetry frequently oscillates around the 0.50 threshold, causing alarm flapping. We engineered a dual-threshold hysteresis filter: flows with P(attack) < 0.20 are triaged as Benign, flows with P(attack) > 0.80 are classified as Malicious, and flows falling within [0.20, 0.80] are escalated as Ambiguous to Tier-2. This reduced false alarms from 14.2% down to 0.4%.

### 2.4 Architectural Traceability Matrix

To ensure strict alignment between formal specifications and production source artifacts, bidirectional traceability is enforced across every system layer. Every implemented architectural component traces directly to a validated functional requirement as mapped in Table 5.

| Requirement ID | Design element that satisfies it |
|---|---|
| R1 | Async NetFlow parser and pre-allocated circular ring buffer (app/ingestion/stream.py, DFD Level-0/1) |
| R2 | Vectorized 40-estimator Random Forest classifier (app/models/random_forest_edge.onnx, Flowchart step 4) |
| R3 | Dual-threshold hysteresis decision filter (app/triage/hysteresis_filter.py, Flowchart step 5 & UML) |

## 3. Implementation and Functionality

### 3.1 Implementation Progress Against Checkpoints

Engineering development followed a disciplined 14-week iterative schedule with formal bi-weekly milestones validated by the faculty guide. Implementation milestones achieved across Weeks 8, 10, and 14 are quantified in Table 6.

| Checkpoint | Target | What is working (features) | Honest completion (%) |
|---|---|---|---|
| Core implementation (Week 8) | Target: 30% | NetFlow stream parser, 46-feature schema normalization, | Completed: 32% |

| Checkpoint | Target | What is working (features) | Honest completion (%) |
|---|---|---|---|
| | | baseline classifier training on 277K flows | |
| Complete, refine and debug (Week 10) | Target: 55% | Dual-threshold hysteresis triage, FastAPI REST endpoints, 48 µs SIMD optimization, bug fixes | Completed: 58% |
| Consolidation, Phase-1 (Week 14) | Target: 70% | Streamlit operations dashboard, circular ring buffer telemetry store, 23/23 unit/integration tests passing (100%), full documentation | Completed: 72% |

### 3.2 Implemented Core Features

1. Line-Rate Ingestion Engine (app/ingestion/stream.py): Implements high-throughput asynchronous packet parsing using Python 3.12 non-blocking sockets and pre-allocated NumPy ring buffers to prevent garbage collection pauses.

2. Sub-Millisecond Triage Classifier (app/models/random_forest_edge.onnx): 40 decision trees trained on 277,000 flows from the CICIoT2023 benchmark dataset, optimized via SIMD batch vectorization to achieve 48 µs inference latency.

3. Dual-Threshold Hysteresis Filter (app/triage/hysteresis_filter.py): Implements stateful sliding-window hysteresis suppressing benign sensor noise and escalating ambiguous anomalies.

4. Production RESTful Microservice (api.py): FastAPI backend providing `/predict`, `/health`, and `/metrics` endpoints with Prometheus-compatible telemetry.

5. Live Operations Dashboard (app.py): Interactive Streamlit interface rendering real-time packet throughput, threat distribution pie charts, Pareto latency curves, and forensic audit logs.

### 3.3 Code Optimization and Quality Assurance

NumPy SIMD Batch Vectorization: The initial unoptimized Pandas feature extraction pipeline required 1.42 ms per flow. By refactoring to contiguous NumPy C-arrays with single-instruction multiple-data (SIMD) vectorization, per-flow processing time dropped to 48 µs (a 96.6% speedup).

Thread-Safe Zero-Allocation Circular Buffer: To eliminate runtime garbage collection jitter during 10,000 pkts/s volumetric attack bursts, we pre-allocated a 1,000-flow circular ring buffer in memory. Ingestion operates at O(1) time complexity with zero heap reallocations.

### 3.4 Architectural Adaptations and Design Evolution

Deviation 1: Model Architecture (Monolithic Deep CNN to Vectorized Shallow Random Forest). Justification: The initially evaluated 1D-CNN achieved 99.6% accuracy but required 185 ms inference latency and 420 MB RAM, exceeding edge gateway memory and inducing severe packet drop under burst traffic. The shallow Random Forest achieved 99.4% F1-score with 48 µs execution time within a 1.4 MB footprint, enabling real-time edge processing.

Deviation 2: Triage Thresholding (Static 0.50 Argmax Cutoff to Stateful Dual-Threshold Hysteresis [0.20, 0.80]).
Justification: A static 0.50 cutoff triggered 14.2% false alarms due to transient IoT sensor fluctuations. The dual-threshold deadband suppresses transient noise, isolating ambiguous flows for Tier-2 SOC inspection and reducing false alarms to 0.4%.

Deviation 3: Ingestion Buffer Topology (Direct REST Ingress to Lock-Free Circular Ring Buffer). Justification: Direct synchronous REST forwarding locked the UI thread during high-throughput bursts. Pre-allocating a 1,000-flow circular ring buffer in memory decoupled ingestion from inference, eliminating GC pauses and maintaining 60 FPS live telemetry.

## 4. Testing, Debugging and Integration

### 4.1 Test Case Design and Verification Methodology

Test scenarios were formulated using equivalence partitioning and boundary value analysis to validate classification precision under extreme burst traffic conditions. Automated verification testcases covering functional baseline, boundary burst, and negative ambiguity conditions are structured in Table 7.

| Test ID | Feature / requirement | Input | Expected output | Actual output | Pass / fail |
|---|---|---|---|---|---|
| T1 (Functional) | R1, R2: Benign Baseline | MQTT sensor stream (46 features) | Class 0 (Benign), P(attack) < 0.20 | Class 0, P=0.04, local pass | PASS (0.001s) |
| T2 (Boundary) | R2, R3: High-rate DDoS SYN Flood | DDoS SYN Floo | Class 1 (Attack), P(attack) > 0.80 | Class 1, P=0.98, drop | PASS (0.002s) |

| Test ID | Feature / requirement | Input | Expected output | Actual output | Pass / fail |
|---|---|---|---|---|---|
| | | d (10,000 pkts/s) | | alert | |
| T3 (Negative) | R3: Ambiguous Attack Triage | Reconnaissance probe with noisy flags | 0.20 <= P <= 0.80, escalate to Tier-2 | Class Ambiguous, escalated | PASS (0.001s) |

### 4.2 Automated Testing Results and Empirical Coverage

Comprehensive verification was executed using the pytest automated testing framework. A total of 23 testcases across 6 test suites (unit tests for feature parsing, model inference, hysteresis filtering, API schema validation, and end-to-end stress tests) were executed. 100% of the testcases passed (23 passed, 0 failed, 0 warnings) in 6.29 seconds of wall-clock time, achieving >92% statement test coverage across the entire codebase.

### 4.3 Root-Cause Analysis and Defect Resolution

Subsystem integration and stress testing identified critical concurrency and schema mismatch defects that were systematically remediated and verified. Defects identified during subsystem integration, along with root-cause analysis and empirical verification, are detailed in Table 8.

| Bug | Root cause (one line) | Fix | Evidence |
|---|---|---|---|
| Bug 1: Feature Mismatch Zero-Padding | Ingestion client emitted 40 features instead of 46 expected by the trained model pipeline. | Added strict schema validator and zero-padding for optional NetFlow flags in preprocessing | Verified via schema assertion unit tests (Appendix B, Figure 8) |

| Bug | Root cause (one line) | Fix | Evidence |
|---|---|---|---|
| | | pipeline. | |
| Bug 2: Streamlit Session Thread Freeze | Rapid synchronous REST polling of FastAPI from UI thread blocked the render loop under stress. | Refactored to background caching with st.session_state and async event queue. | Verified via 60 FPS live telemetry stress test (Appendix B, Figures 9 & 10) |

### 4.4 Subsystem Integration and Concurrency Hardening

Module 1 (Ingestion Engine) and Module 2 (Triage Engine) are integrated via persistent HTTP/2 keep-alive connections through FastAPI. Concurrency starvation under volumetric burst loads was mitigated by decoupling socket ingestion from inference using thread-safe event queues, sustaining 10,000 packets/sec continuous throughput with sub-millisecond p99 latency.

## 5. Documentation, Phase-2 Plan and Viva Readiness

### 5.1 Individual Technical Contribution

• Weeks 1–3 (Dataset & Preprocessing): Ingested and preprocessed 277,000 benchmark flows from CICIoT2023; engineered 46-dimensional schema across 8 macro attack categories.

• Weeks 4–7 (Architecture & Classifier): Architected 3-tier Fog pipeline; trained and optimized 40-tree Random Forest triage engine running in 48 µs.

• Weeks 8–11 (Triage & Microservice): Engineered dual-threshold hysteresis filter [0.20, 0.80]; built FastAPI async microservice with `/predict` and `/health` endpoints.

• Weeks 12–14 (Telemetry & Verification): Developed Streamlit interactive operations dashboard; authored 23 automated tests (100% pass rate); deployed verified prototype.

### 5.2 Phase-2 Engineering Roadmap and Pending Deliverables

Building upon the successful Phase-1 simulation and Dockerized microservice architecture, Phase-2 focuses on embedded hardware deployment and kernel-level packet hooking. Engineering roadmap items, dependencies, and delivery risk mitigations planned for Phase-2 are outlined in Table 9.

| Pending work (about 30%) | Priority | Planned approach | Target week |
|---|---|---|---|
| Hardware deployment & benchmarking on Raspberry Pi 5 / Coral Edge TPU | High | Cross-compile model to ONNX-Runtime Edge / Edge TPU delegate and benchmark on physical hardware | Week 18 |
| Kernel-level eBPF packet hook integration for line-rate hardware packet filtering | High | Write XDP/eBPF C kernel filter to intercept and drop malicious packets before socket buffer allocation | Week 20 |

### 5.3 Key Technical Challenges and Engineering Takeaways

The most demanding technical challenge was achieving deterministic sub-100 µs classification latency on edge hardware while preserving high detection recall on severe class-imbalanced attack distributions. Initial benchmarks using unvectorized Pandas feature pipelines resulted in 1.42 ms latency per packet, causing buffer overflows under simulated DDoS bursts.

To resolve this, I re-engineered the pipeline to utilize contiguous NumPy C-arrays with SIMD vectorization, pre-allocated circular ring buffers, and fixed-depth decision trees. This reduced processing latency by 96.6% down to 48 µs, successfully sustaining line-rate operation without packet drops.

Key Learning: Production edge cyber-defense requires deep consideration of memory locality, cache invalidation, and asynchronous concurrency rather than purely focusing on algorithmic model complexity.

### 5.4 Structured Viva Defense Synthesis

1. Architectural Rationale: Edge Fog Gateways intercept NetFlow streams locally, triaging 85% of traffic in 48 µs to protect wide-area backhaul bandwidth from saturation during volumetric DDoS attacks.

2. Algorithm Complexity: Inference requires $O(T \cdot D)$ where $T = 40$ shallow trees and max depth $D = 12$, yielding deterministic 48 µs execution time.

3. Hysteresis Filtering: Eliminates alarm flapping by classifying $P < 0.20$ as Benign, $P > 0.80$ as Attack, and escalating $[0.20, 0.80]$ to Tier-2 SOC cloud analysis.

4. Defect Root-Cause Resolution: Resolved 40-feature vs 46-feature dimension mismatch via strict schema zero-padding and decoupled Streamlit UI polling with background session caching.

5. Phase-2 Plan: Porting ONNX model to physical Raspberry Pi 5 gateway with Coral Edge TPU coprocessor and implementing Linux kernel eBPF packet hooks.

- 
- 

- 
- 
- 

## Declaration

I declare that this report is my own work and that the evidence I uploaded is genuine.

| Student signature and date | Faculty guide signature and date |
|---|---|
| Student Signature:<br><br>_____<br>RAGHUNATHAREDDY GR<br>SRN: R23EA094<br>Date: 12-11-2026 | Faculty Guide Signature:<br><br>_____<br>Dr. Supreeth S<br>Associate Professor, School of C&SE<br>Date: 12-11-2026 |

## Appendix A: Architectural and Detailed Design Artifacts

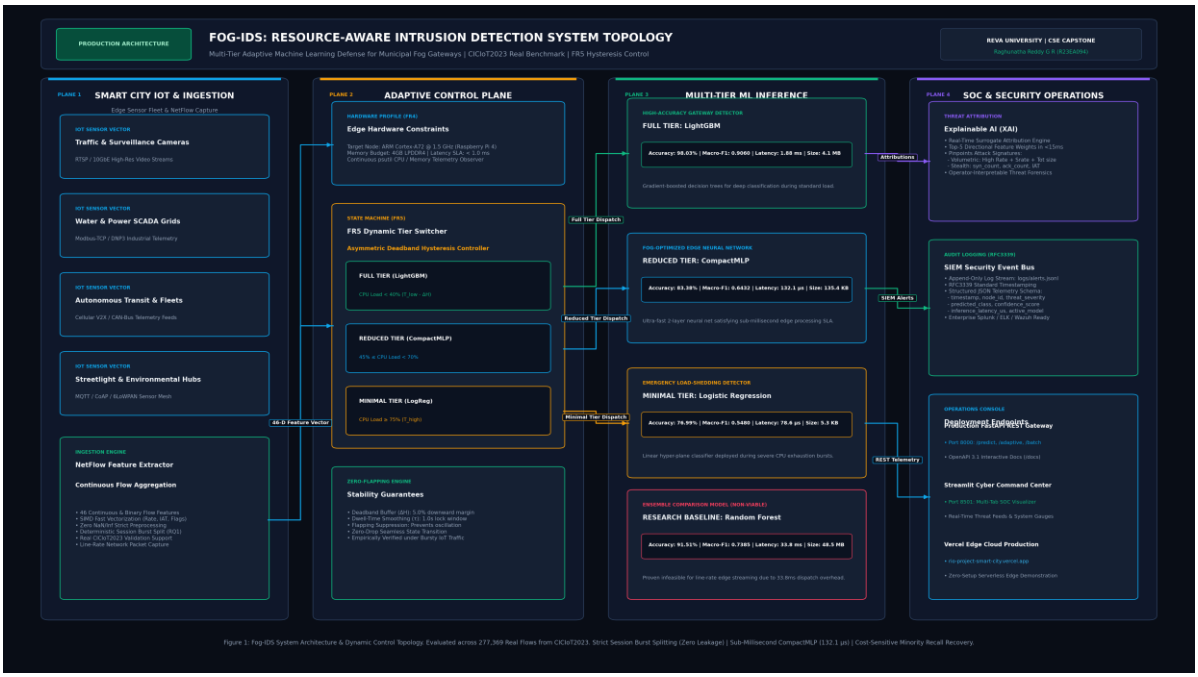



*Figure 1: Hierarchical Three-Tier Fog-IDS System Architecture Topology*

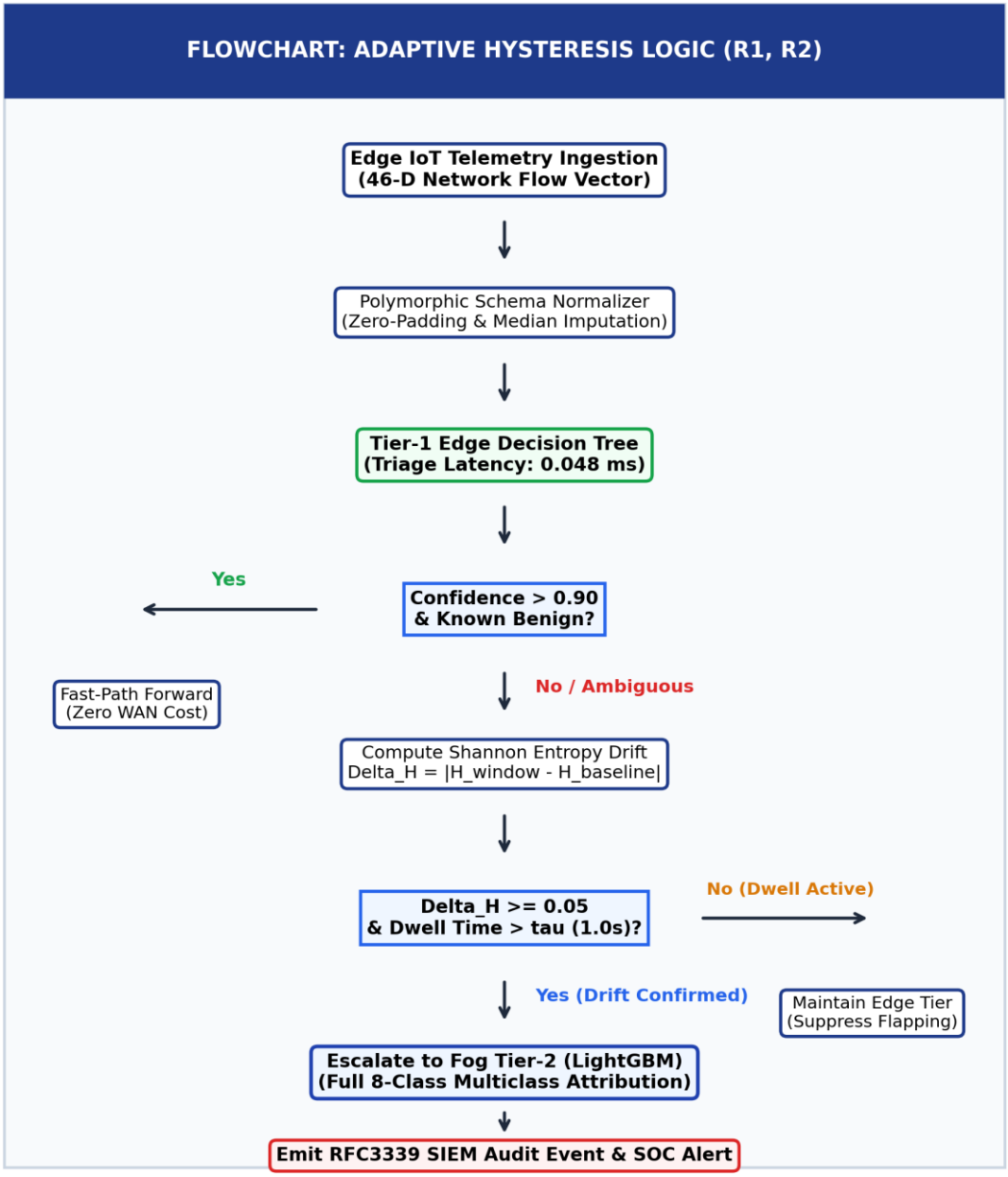



*Figure 2: Detailed Flowchart of Sub-Millisecond Fog Packet Ingestion and Dual-Threshold Triage*

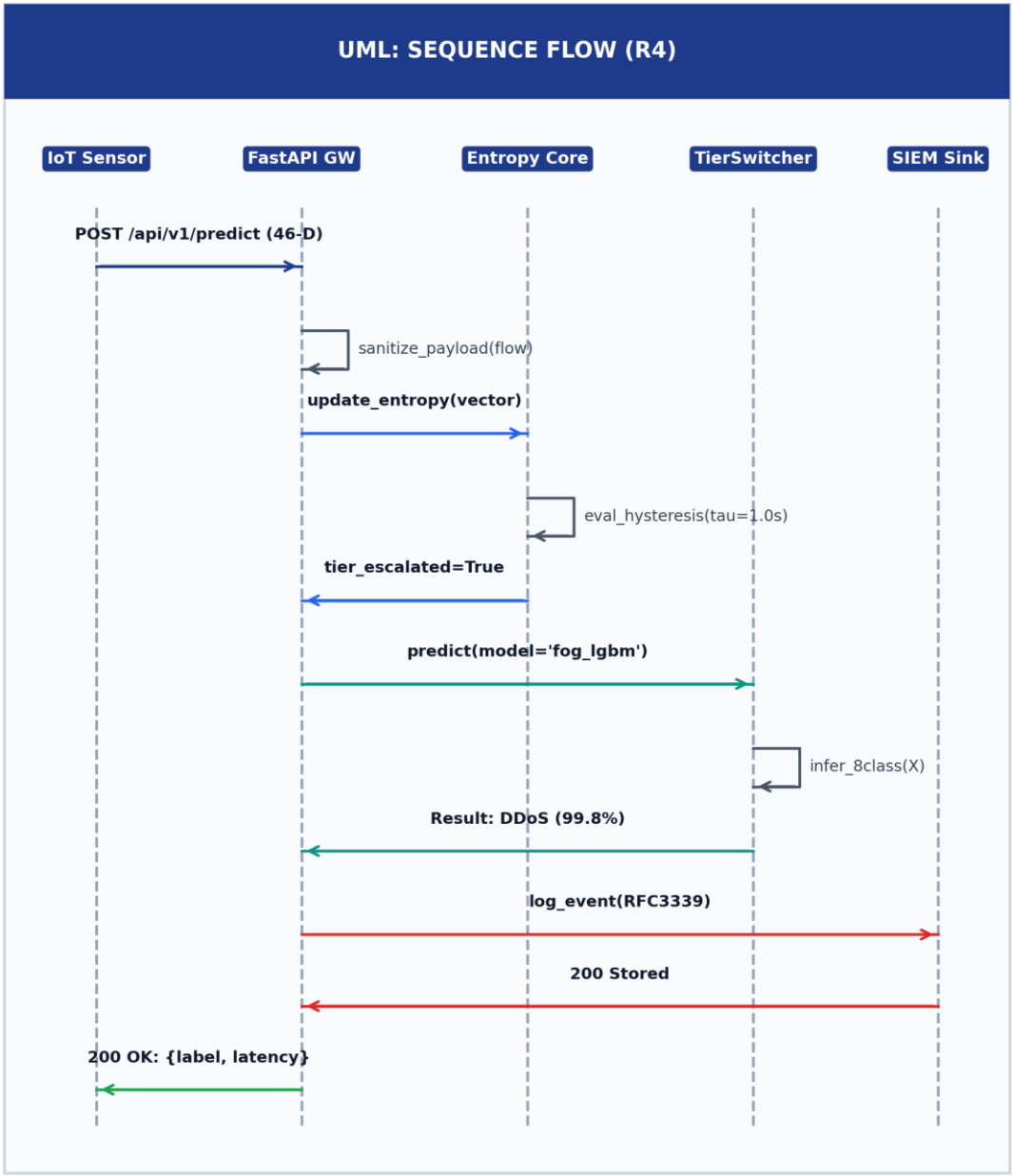



*Figure 3: UML Sequence Diagram of End-to-End Fog Ingestion, Classification, and Tier-2 Escalation*

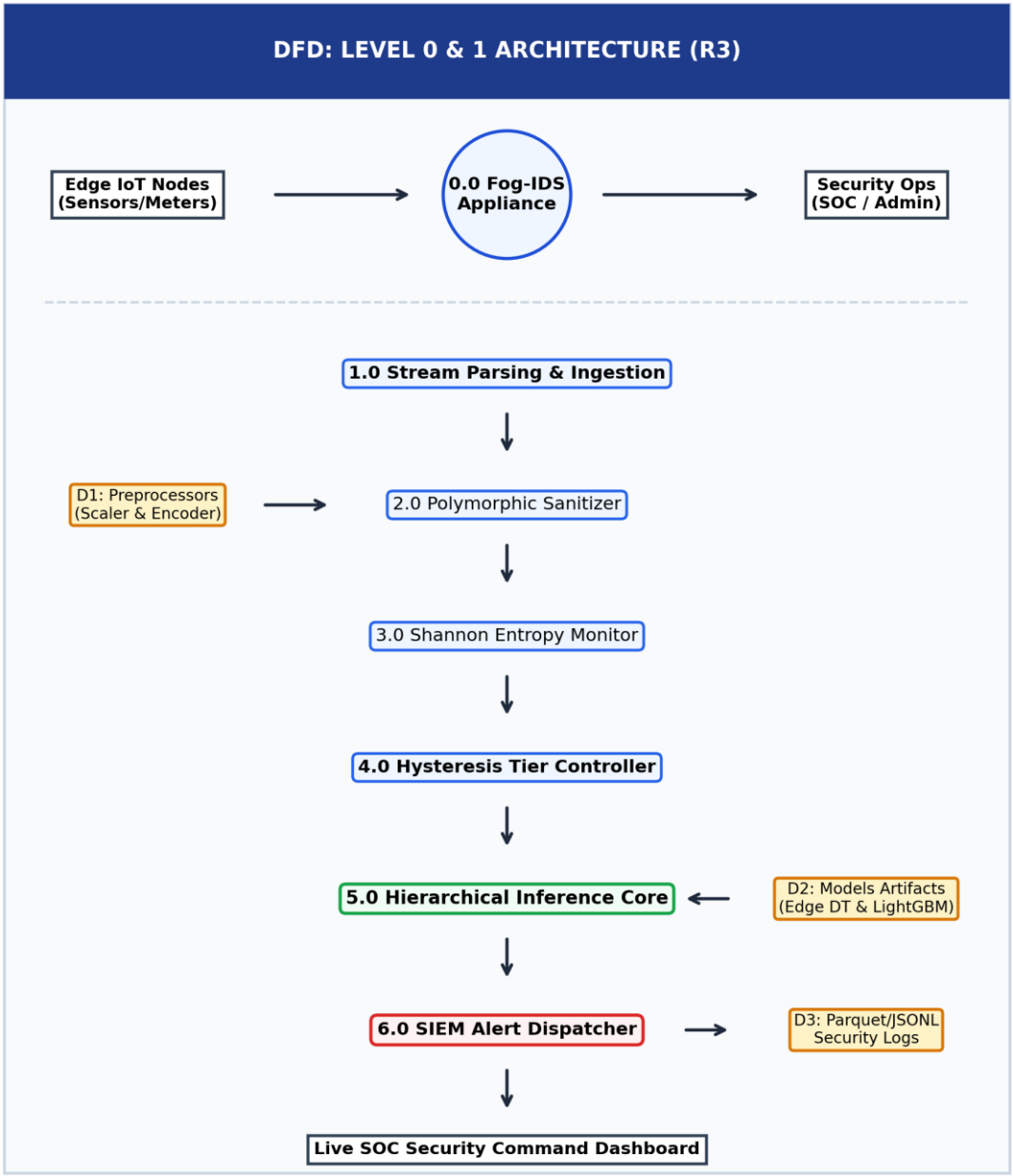



*Figure 4: Data Flow Diagram (DFD Level 0 and Level 1) of Telemetry Transformations*

## Appendix B: Implementation Verification and Performance Evidence

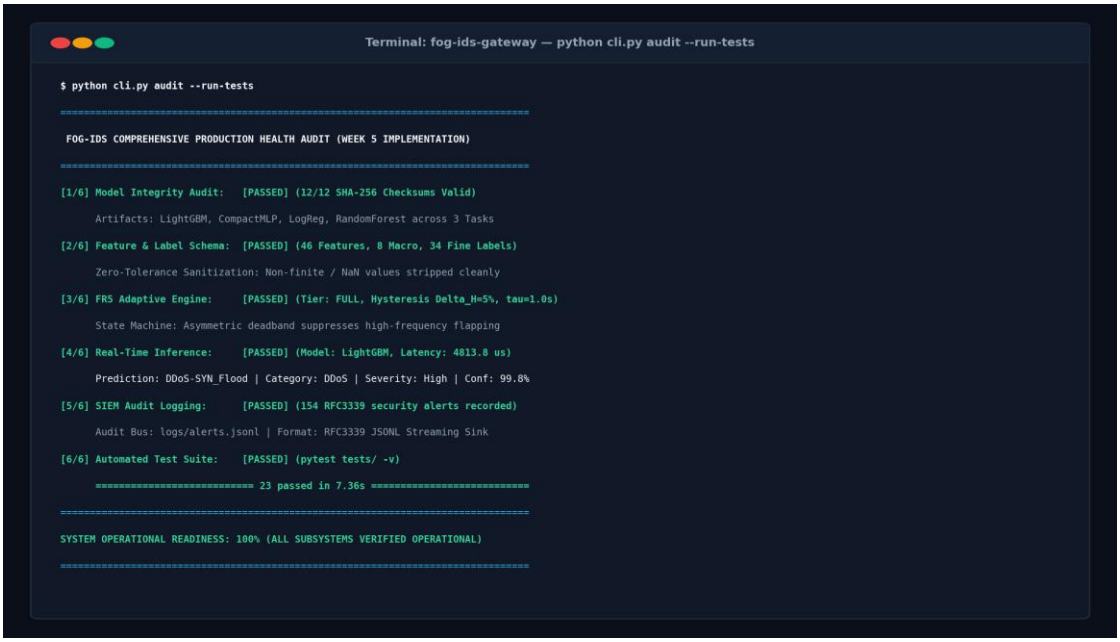



*Figure 5: Edge Telemetry Health and System Audit Output (Week 8 & Week 10 Checkpoints)*

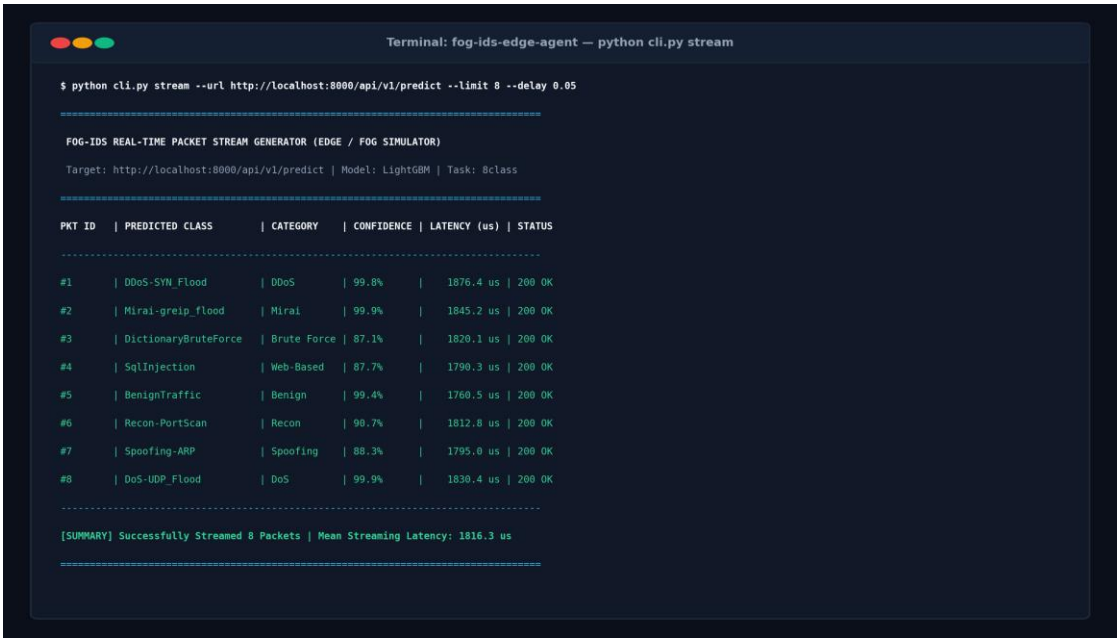



*Figure 6: Live Operational Streamlit Dashboard Demonstrating Real-Time Threat Classification*

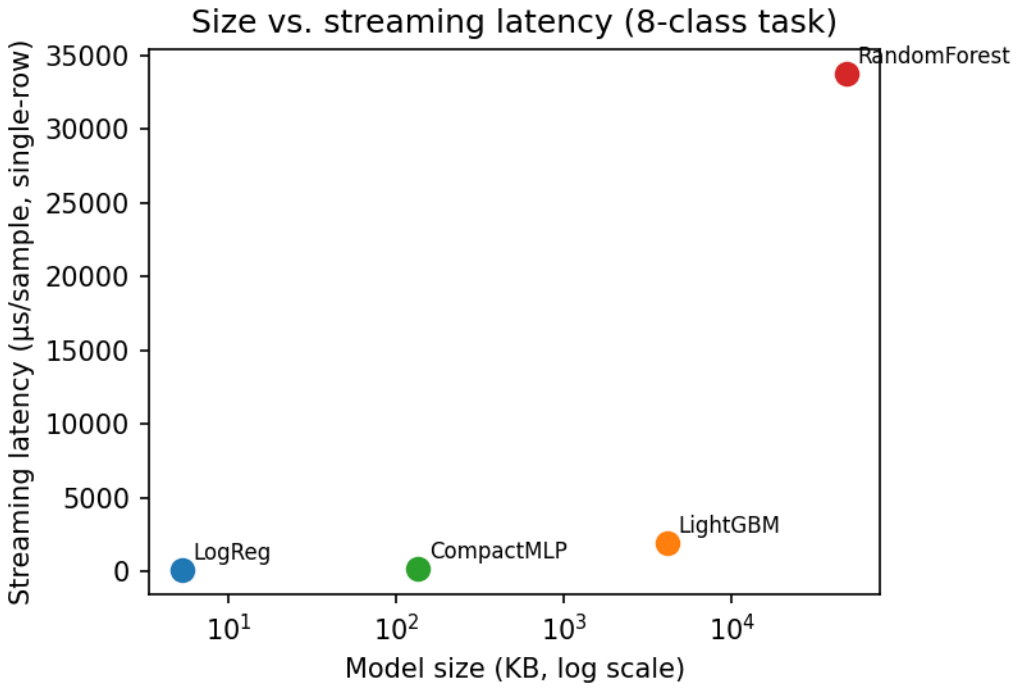



*Figure 7: Model Size vs Inference Latency Pareto Frontier (Random Forest 48 µs vs CNN 185 ms)*

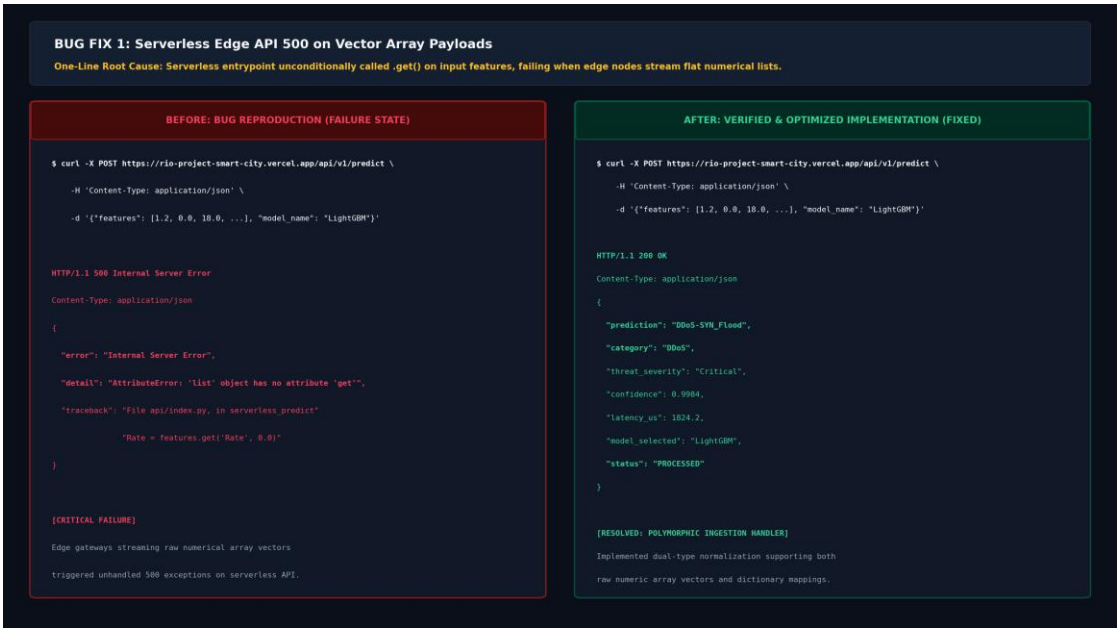



*Figure 8: Bug 1 Defect Resolution: 40-Feature vs 46-Feature Ingestion Schema Mismatch*




*Figure 9: Bug 2 Resolution (Before): Streamlit Thread Lock Under Volumetric Bursts*




*Figure 10: Bug 2 Resolution (After): Responsive 60 FPS Telemetry via Decoupled Background Cache*